

midterm_test

October 1, 2025

```
[2]: import numpy as np # Xử lý mảng
import pandas as pd # Xử lý dữ liệu
import matplotlib.pyplot as plt # Vẽ biểu đồ
import seaborn as sns # Vẽ biểu đồ nâng cao
from sklearn.model_selection import train_test_split # Chia dữ liệu
# Regression models
from sklearn.linear_model import LinearRegression, Ridge, Lasso
from sklearn.tree import DecisionTreeRegressor
from sklearn.ensemble import RandomForestRegressor, GradientBoostingRegressor
from sklearn.svm import SVR
from sklearn.neighbors import KNeighborsRegressor
# Classification models
from sklearn.linear_model import LogisticRegression
from sklearn.tree import DecisionTreeClassifier
from sklearn.ensemble import RandomForestClassifier, GradientBoostingClassifier
from sklearn.svm import SVC
from sklearn.neighbors import KNeighborsClassifier
from sklearn.naive_bayes import GaussianNB
# Metrics
from sklearn.metrics import mean_absolute_error, mean_squared_error, r2_score # ↪
↪Regression metrics
from sklearn.metrics import accuracy_score, precision_score, recall_score, ↪
↪f1_score, confusion_matrix # Classification metrics
import joblib # Lưu mô hình và tái sử dụng
import os # Xử lý tệp và thư mục
import warnings # Xử lý cảnh báo
warnings.filterwarnings("ignore") # Bỏ qua cảnh báo không cần thiết
from sklearn.metrics import classification_report, accuracy_score, roc_auc_score
# Xử lý dữ liệu
from sklearn.impute import SimpleImputer # Xử lý giá trị thiếu
from sklearn.preprocessing import StandardScaler, OneHotEncoder # Chuẩn hóa và ↪
↪mã hóa
from sklearn.compose import ColumnTransformer # Biến đổi cột
from sklearn.pipeline import Pipeline # Tạo pipeline
from sklearn.metrics import classification_report # Báo cáo phân loại
from sklearn.preprocessing import LabelEncoder # Mã hóa nhãn
```

```
[3]: data = pd.read_csv('data\healthcare_dataset.csv')
data.head()
```

```
[3]:
```

	Name	Age	Gender	Blood Type	Medical Condition	Date of Admission	\
0	Bobby JacksOn	30	Male	B-	Cancer	2024-01-31	
1	LesLie TErRy	62	Male	A+	Obesity	2019-08-20	
2	DaNnY sMitH	76	Female	A-	Obesity	2022-09-22	
3	andrEw waTtS	28	Female	O+	Diabetes	2020-11-18	
4	adrIENNE bEll	43	Female	AB+	Cancer	2022-09-19	

	Doctor	Hospital	Insurance Provider	\
0	Matthew Smith	Sons and Miller	Blue Cross	
1	Samantha Davies	Kim Inc	Medicare	
2	Tiffany Mitchell	Cook PLC	Aetna	
3	Kevin Wells	Hernandez Rogers and Vang,	Medicare	
4	Kathleen Hanna	White-White	Aetna	

	Billing Amount	Room Number	Admission Type	Discharge Date	Medication	\
0	18856.281306	328	Urgent	2024-02-02	Paracetamol	
1	33643.327287	265	Emergency	2019-08-26	Ibuprofen	
2	27955.096079	205	Emergency	2022-10-07	Aspirin	
3	37909.782410	450	Elective	2020-12-18	Ibuprofen	
4	14238.317814	458	Urgent	2022-10-09	Penicillin	

	Test Results
0	Normal
1	Inconclusive
2	Normal
3	Abnormal
4	Abnormal

```
[4]: data.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 55500 entries, 0 to 55499
Data columns (total 15 columns):
#   Column                Non-Null Count  Dtype
---  -
0   Name                   55500 non-null  object
1   Age                    55500 non-null  int64
2   Gender                 55500 non-null  object
3   Blood Type             55500 non-null  object
4   Medical Condition      55500 non-null  object
5   Date of Admission      55500 non-null  object
6   Doctor                 55500 non-null  object
7   Hospital               55500 non-null  object
8   Insurance Provider     55500 non-null  object
9   Billing Amount          55500 non-null  float64
```

```

10 Room Number          55500 non-null  int64
11 Admission Type       55500 non-null  object
12 Discharge Date       55500 non-null  object
13 Medication           55500 non-null  object
14 Test Results         55500 non-null  object
dtypes: float64(1), int64(2), object(12)
memory usage: 6.4+ MB

```

```

[13]: X = data[['Medical Condition', 'Age', 'Admission Type', 'Billing Amount',
↳ 'Medication', 'Blood Type']]
y = data['Test Results']

```

```

[6]: le = LabelEncoder()
y_encoded = le.fit_transform(y)

```

```

[7]: numeric_features = X.select_dtypes(include=['int64', 'float64']).columns
# numeric_features = ["Age", "Billing Amount"]
categorical_features = X.select_dtypes(include=['object']).columns

numeric_transformer = Pipeline(steps=[
    ("imputer", SimpleImputer(strategy="mean")),
    ("scaler", StandardScaler())
])

categorical_transformer = Pipeline(steps=[
    ("imputer", SimpleImputer(strategy="most_frequent")),
    ("encoder", OneHotEncoder(handle_unknown="ignore"))
])

preprocessor = ColumnTransformer(
    transformers=[
        ("num", numeric_transformer, numeric_features),
        ("cat", categorical_transformer, categorical_features)
    ]
)

```

```

[8]: X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2,
↳ random_state=42, stratify=y_encoded)

```

```

[12]: models = {
    "Logistic Regression": LogisticRegression(max_iter=1000),
    "Decision Tree": DecisionTreeClassifier(random_state=42)
}

for name, model in models.items():
    clf = Pipeline(steps=[
        ("preprocessor", preprocessor),

```

```

        ("classifier", model)
    ])
    clf.fit(X_train, y_train)
    y_pred = clf.predict(X_test)
    y_proba = clf.predict_proba(X_test)
    print(f"==== {name} =====")
    print(classification_report(y_test, y_pred))
    print("ROC-AUC:", roc_auc_score(y_test, y_proba, multi_class="ovr"), "\n")

```

==== Logistic Regression =====

	precision	recall	f1-score	support
Abnormal	0.33	0.47	0.39	3726
Inconclusive	0.33	0.19	0.24	3671
Normal	0.34	0.33	0.33	3703
accuracy			0.33	11100
macro avg	0.33	0.33	0.32	11100
weighted avg	0.33	0.33	0.32	11100

ROC-AUC: 0.497291858399791

==== Decision Tree =====

	precision	recall	f1-score	support
Abnormal	0.40	0.39	0.40	3726
Inconclusive	0.39	0.39	0.39	3671
Normal	0.40	0.40	0.40	3703
accuracy			0.39	11100
macro avg	0.39	0.39	0.39	11100
weighted avg	0.39	0.39	0.39	11100

ROC-AUC: 0.5447906123251882